

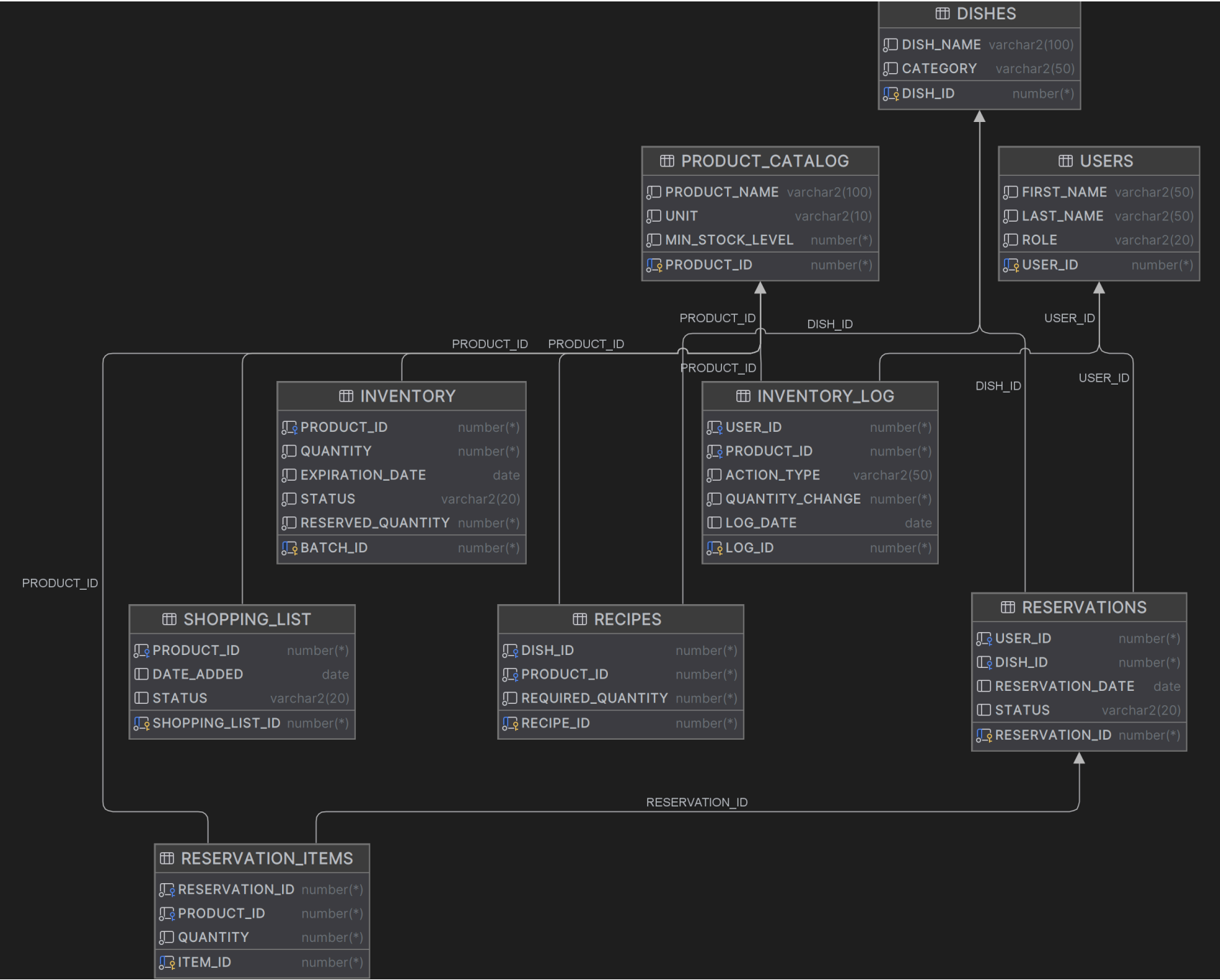
<style> body { font-size: 8pt; } </style>

Projekt - spizarnia

Imiona i nazwiska: Iga Szaflik, Katarzyna Piskorz

repozytorium: https://github.com/kassimma123/BD_2_Piskorz_Szaflik.git

Schemat Bazy danych



Users (Użytkownicy)

Tabela przechowuje informacje o użytkownikach systemu (np. domownikach, kucharzach), którzy mogą modyfikować stany magazynowe lub dokonywać rezerwacji posiłków.

```
CREATE TABLE Users (  
  User_ID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  First_Name VARCHAR2(50) NOT NULL,  
  Last_Name VARCHAR2(50) NOT NULL,  
  Role VARCHAR2(20) NOT NULL  
);
```

Nazwa kolumny	Typ danych	Ograniczenia	Opis
User_ID	NUMBER	PRIMARY KEY, IDENTITY	Unikalny identyfikator użytkownika (Klucz główny, autoinkrementacja).
First_Name	VARCHAR2(50)	NOT NULL	Imię użytkownika.
Last_Name	VARCHAR2(50)	NOT NULL	Nazwisko użytkownika.
Role	VARCHAR2(20)	NOT NULL	Rola w systemie (np. 'ADMIN', 'USER', 'CHEF').

Product_Catalog (Katalog Produktów)

Słownik wszystkich produktów, które mogą pojawić się w spiżarni lub w przepisach. Definiuje podstawowe właściwości produktu.

```
CREATE TABLE Product_Catalog (  
  Product_ID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  Product_Name VARCHAR2(100) NOT NULL,  
  Unit VARCHAR2(10) NOT NULL,  
  Min_Stock_Level NUMBER NOT NULL,  
);
```

Nazwa kolumny	Typ danych	Ograniczenia	Opis
Product_ID	NUMBER	PRIMARY KEY, IDENTITY	Unikalny identyfikator produktu.
Product_Name	VARCHAR2(100)	NOT NULL	Nazwa produktu (np. "Mleko 3,2%", "Mąka pszenna").
Unit	VARCHAR2(10)	NOT NULL	Jednostka miary (np. 'kg', 'litr', 'szt').
Min_Stock_Level	NUMBER	NOT NULL	Minimalny próg zapasu. Spadek poniżej tej wartości może generować wpis na listę zakupów.

Inventory (Spizarnia / Stany Magazynowe)

Tabela przechowuje fizyczne stany magazynowe. Rekordy reprezentują konkretne partie produktów.

```
CREATE TABLE Inventory (  
  Batch_ID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  Product_ID NUMBER NOT NULL,  
  Quantity NUMBER NOT NULL,  
  Expiration_Date DATE NOT NULL,  
  Status VARCHAR2(20) NOT NULL,  
  FOREIGN KEY (Product_ID) REFERENCES Product_Catalog(Product_ID),  
  Reserved_Quantity NUMBER DEFAULT 0 NOT NULL  
);
```

Nazwa kolumny	Typ danych	Ograniczenia	Opis
Batch_ID	NUMBER	PRIMARY KEY, IDENTITY	Unikalny identyfikator partii produktu.
Product_ID	NUMBER	FOREIGN KEY, NOT NULL	Odniesienie do produktu w Product_Catalog.
Quantity	NUMBER	NOT NULL	Aktualna, fizyczna ilość produktu w tej partii.
Expiration_Date	DATE	NOT NULL	Data ważności danej partii.

Nazwa kolumny	Typ danych	Ograniczenia	Opis
Status	VARCHAR2(20)	NOT NULL	Status partii (np. 'FRESH', 'EXPIRED', 'OPENED').
Reserved_Quantity	NUMBER	NOT NULL, DEFAULT 0	Ilość produktu z tej partii zarezerwowana na poczet zaplanowanych dań.

Dishes (Dania w menu)

Słownik dostępnych dań, które można przygotować korzystając z produktów w szpitalu.

```
CREATE TABLE Dishes (  
  Dish_ID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  Dish_Name VARCHAR2(100) NOT NULL,  
  Category VARCHAR2(50) NOT NULL  
);
```

Nazwa kolumny	Typ danych	Ograniczenia	Opis
Dish_ID	NUMBER	PRIMARY KEY, IDENTITY	Unikalny identyfikator dania.
Dish_Name	VARCHAR2(100)	NOT NULL	Nazwa dania (np. "Spaghetti Bolognese").
Category	VARCHAR2(50)	NOT NULL	Kategoria dania (np. 'ZUPA', 'DANIE GŁÓWNE', 'DESER').

Recipes (Przepisy)

Tabela łącząca dania z produktami. Określa listę składników (oraz ich ilości) wymaganych do przygotowania konkretnego dania.

```
CREATE TABLE Recipes (  
  Recipe_ID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  Dish_ID NUMBER NOT NULL,  
  Product_ID NUMBER NOT NULL,  
  Required_Quantity NUMBER NOT NULL,
```

```
FOREIGN KEY (Dish_ID) REFERENCES Dishes(Dish_ID),
FOREIGN KEY (Product_ID) REFERENCES Product_Catalog(Product_ID)
);
```

Nazwa kolumny	Typ danych	Ograniczenia	Opis
Recipe_ID	NUMBER	PRIMARY KEY, IDENTITY	Unikalny identyfikator wpisu w przepisie.
Dish_ID	NUMBER	FOREIGN KEY, NOT NULL	Odniesienie do dania w tabeli Dishes.
Product_ID	NUMBER	FOREIGN KEY, NOT NULL	Odniesienie do wymaganego produktu (Product_Catalog).
Required_Quantity	NUMBER	NOT NULL	Ilość produktu potrzebna do przygotowania porcji dania.

Shopping_List (Lista Zakupów)

Rejestr brakujących produktów, które należy dokupić.

```
CREATE TABLE Shopping_List (
  Shopping_List_ID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Product_ID NUMBER NOT NULL,
  Date_Added DATE DEFAULT SYSDATE,
  Status VARCHAR2(20) DEFAULT 'TO_BUY',
  FOREIGN KEY (Product_ID) REFERENCES Product_Catalog(Product_ID)
);
```

Nazwa kolumny	Typ danych	Ograniczenia	Opis
Shopping_List_ID	NUMBER	PRIMARY KEY, IDENTITY	Unikalny identyfikator pozycji na liście.
Product_ID	NUMBER	FOREIGN KEY, NOT NULL	Odniesienie do produktu z katalogu.
Date_Added	DATE	DEFAULT SYSDATE	Data dodania produktu na listę zakupów.

Nazwa kolumny	Typ danych	Ograniczenia	Opis
Status	VARCHAR2(20)	DEFAULT 'TO_BUY'	Status pozycji (np. 'TO_BUY', 'BOUGHT', 'IGNORED').

Reservations (Rezerwacje)

Nagłówek rezerwacji. Pozwala zaplanować przygotowanie konkretnego dania, co wiąże się z alokacją składników w spiżarni.

```
CREATE TABLE Reservations (  
  Reservation_ID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  User_ID NUMBER NOT NULL,  
  Dish_ID NUMBER,  
  Reservation_Date DATE DEFAULT SYSDATE,  
  Status VARCHAR2(20) DEFAULT 'ACTIVE',  
  FOREIGN KEY (User_ID) REFERENCES Users(User_ID),  
  FOREIGN KEY (Dish_ID) REFERENCES Dishes(Dish_ID)  
);
```

Nazwa kolumny	Typ danych	Ograniczenia	Opis
Reservation_ID	NUMBER	PRIMARY KEY, IDENTITY	Unikalny identyfikator rezerwacji.
User_ID	NUMBER	FOREIGN KEY, NOT NULL	Użytkownik dokonujący rezerwacji/planujący posiłek.
Dish_ID	NUMBER	FOREIGN KEY	Danie, które ma zostać przygotowane.
Reservation_Date	DATE	DEFAULT SYSDATE	Data utworzenia rezerwacji (lub data planowanego posiłku).
Status	VARCHAR2(20)	DEFAULT 'ACTIVE'	Status rezerwacji (np. 'ACTIVE', 'COMPLETED', 'CANCELLED').

Reservation_Items (Szczegóły Rezerwacji)

Tabela przechowująca szczegóły dotyczące produktów i ich ilości zablokowanych na poczet danej rezerwacji.

```
CREATE TABLE Reservation_Items (  
  Item_ID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  Reservation_ID NUMBER NOT NULL,  
  Product_ID NUMBER NOT NULL,  
  Quantity NUMBER NOT NULL,  
  FOREIGN KEY (Reservation_ID) REFERENCES Reservations(Reservation_ID),  
  FOREIGN KEY (Product_ID) REFERENCES Product_Catalog(Product_ID)  
);
```

Nazwa kolumny	Typ danych	Ograniczenia	Opis
Item_ID	NUMBER	PRIMARY KEY, IDENTITY	Unikalny identyfikator pozycji rezerwacji.
Reservation_ID	NUMBER	FOREIGN KEY, NOT NULL	Odniesienie do nagłówka rezerwacji w Reservations.
Product_ID	NUMBER	FOREIGN KEY, NOT NULL	Rezerwowany produkt.
Quantity	NUMBER	NOT NULL	Dokładna zablokowana ilość danego produktu.

Inventory_Log (Historia)

Tabela logów śledząca wszelkie operacje wykonywane na magazynie.


```
CREATE TABLE Inventory_Log (  
  Log_ID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  User_ID NUMBER NOT NULL,  
  Product_ID NUMBER NOT NULL,  
  Action_Type VARCHAR2(50) NOT NULL,  
  Quantity_Change NUMBER NOT NULL,  
  Log_Date DATE DEFAULT SYSDATE,  
  FOREIGN KEY (User_ID) REFERENCES Users(User_ID),  
  FOREIGN KEY (Product_ID) REFERENCES Product_Catalog(Product_ID)  
);
```

Nazwa kolumny	Typ danych	Ograniczenia	Opis
Log_ID	NUMBER	PRIMARY KEY, IDENTITY	Unikalny identyfikator wpisu w logach.
User_ID	NUMBER	FOREIGN KEY, NOT NULL	Użytkownik, który wykonał akcję w spiżarni.
Product_ID	NUMBER	FOREIGN KEY, NOT NULL	Produkt, którego dotyczy zmiana.
Action_Type	VARCHAR2(50)	NOT NULL	Typ operacji (np. 'ADDED', 'REMOVED', 'SPOILED', 'RESERVED').
Quantity_Change	NUMBER	NOT NULL	Wielkość zmiany (+/- ilość dodana lub ujęta).
Log_Date	DATE	DEFAULT SYSDATE	Dokładny czas wykonania operacji.

Zabezpieczenie

Aby system był odporny na błędy i unikał sytuacji niemożliwych w świecie rzeczywistym, zastosowałam ograniczenia typu CHECK CONSTRAINT. Baza danych automatycznie odrzuci każdą próbę wprowadzenia wartości mniejszej niż zero dla ilości produktów oraz rezerwacji.

```
-- Zabezpieczenie spiżarni  
ALTER TABLE Inventory ADD CONSTRAINT chk_inv_quantity CHECK (Quantity >= 0);  
ALTER TABLE Inventory ADD CONSTRAINT chk_inv_reserved CHECK (Reserved_Quantity >= 0);
```

```
-- Zabezpieczenie przepisów
ALTER TABLE Recipes ADD CONSTRAINT chk_recipes_req_qty CHECK (Required_Quantity > 0);

-- Zabezpieczenie szczegółów rezerwacji
ALTER TABLE Reservation_Items ADD CONSTRAINT chk_res_items_qty CHECK (Quantity > 0);
```

Triggery

Wyzwalacz historii zmian: TRG_Inventory_History

```
CREATE OR REPLACE TRIGGER TRG_Inventory_History
AFTER INSERT OR UPDATE OR DELETE ON Inventory
FOR EACH ROW
DECLARE
    v_user_id NUMBER := 1;
    v_action VARCHAR2(50);
    v_qty_change NUMBER;
    v_product_id NUMBER;
BEGIN
    IF INSERTING THEN
        v_action := 'ADDED_NEW_BATCH';
        v_qty_change := :NEW.Quantity;
        v_product_id := :NEW.Product_ID;

    ELSIF UPDATING THEN
        v_product_id := :NEW.Product_ID;

    -- zmiana całkowitej ilości
    IF :NEW.Quantity != :OLD.Quantity THEN
        v_action := 'QUANTITY_CHANGED';
        v_qty_change := :NEW.Quantity - :OLD.Quantity;
    -- zmiana zarezerwowanej ilości
    ELSIF :NEW.Reserved_Quantity != :OLD.Reserved_Quantity THEN
        v_action := 'RESERVATION_UPDATED';
```

```
        v_qty_change := :NEW.Reserved_Quantity - :OLD.Reserved_Quantity;
    ELSE
        v_action := 'STATUS_UPDATED (' || :NEW.Status || ')';
        v_qty_change := 0;
    END IF;
ELSIF DELETING THEN
    v_action := 'REMOVED_BATCH';
    v_qty_change := -:OLD.Quantity;
    v_product_id := :OLD.Product_ID;
END IF;

INSERT INTO Inventory_log (User_ID, Product_ID, Action_Type, Quantity_Change, Log_Date)
VALUES (v_user_id, v_product_id, v_action, v_qty_change, SYSDATE);
END;
```

- Zapewnienie pełnej historii operacji magazynowych (kto, kiedy i co zmodyfikował w spizarni).
- Wyzwalacz reaguje na każdą operację na tabeli **Inventory**. W zależności od rodzaju operacji, automatycznie oblicza różnicę w stanach magazynowych lub rezerwacjach i zapisuje te dane do tabeli **Inventory_Log** wraz z aktualną datą systemową (**SYSDATE**).
- Uniemożliwia ręczną zmianę stanów magazynowych "poza plecami" systemu, co jest kluczowe w zarządzaniu kosztami restauracji.

Wyzwalacz automatycznych zakupów: **TRG_Auto_Restock**

```
CREATE OR REPLACE TRIGGER TRG_Auto_Restock
AFTER UPDATE OF Quantity, Reserved_Quantity ON Inventory
FOR EACH ROW
DECLARE
    PRAGMA AUTONOMOUS_TRANSACTION;
    v_Min_Stock NUMBER;
    v_Total_Current_Available NUMBER;
    v_Already_On_List NUMBER;
BEGIN
    SELECT Min_Stock_Level INTO v_Min_Stock FROM Product_Catalog WHERE Product_ID = :NEW.Product_ID;

    SELECT NVL(SUM(Quantity - Reserved_Quantity), 0) INTO v_Total_Current_Available
```

```
FROM Inventory
WHERE Product_ID = :NEW.Product_ID AND Status = 'AVAILABLE' AND Expiration_Date >= TRUNC(SYSDATE);

SELECT COUNT(*) INTO v_Already_On_List
FROM Shopping_List
WHERE Product_ID = :NEW.Product_ID AND Status = 'TO_BUY';

IF v_Total_Current_Available < v_Min_Stock AND v_Already_On_List = 0 THEN
    INSERT INTO Shopping_List (Product_ID, Date_Added, Status)
    VALUES (:NEW.Product_ID, SYSDATE, 'TO_BUY');
END IF;

COMMIT;
END;
/
```

- **Automatyzacja zarządzania zapasami:** Wyzwalacz w locie sprawdza stan spiżarni przy każdej zmianie ilości produktu.
- Jeśli suma wolnego towaru (dostępnego i niezarezerwowanego) spadnie poniżej ustalonego progu bezpieczeństwa (**Min_Stock_Level**), system samodzielnie wpisuje brakujący produkt na listę zakupów (**Shopping_List**).
- Trigger zabezpiecza nas przed dublowaniem zamówień – upewnia się najpierw, czy produkt nie widnieje już na liście ze statusem **TO_BUY**. Dzięki autonomicznej transakcji (**PRAGMA AUTONOMOUS_TRANSACTION**) logowanie braku dokonuje się płynnie, niezależnie od innych zdarzeń na głównej transakcji.

Procedury

Procedura rozliczania i zwalniania rezerwacji: **Resolve_Reservation**

```
CREATE OR REPLACE PROCEDURE Resolve_Reservation(
    p_reservation_id IN NUMBER,
    p_action IN VARCHAR2
)
IS
    v_status VARCHAR2(20);
    v_qty_to_process NUMBER;
```

```
v_deduct NUMBER;

-- produkty w rezerwacji
CURSOR c_items IS
    SELECT Product_ID, Quantity
    FROM Reservation_Items
    WHERE Reservation_ID = p_reservation_id;

CURSOR c_inventory(p_prod NUMBER) IS
    SELECT Batch_ID, Quantity, Reserved_Quantity
    FROM Inventory
    WHERE Product_ID = p_prod AND Reserved_Quantity > 0
    ORDER BY Expiration_Date ASC
    FOR UPDATE; -- blokowanie

BEGIN
    SELECT Status INTO v_status FROM Reservations WHERE Reservation_id = p_reservation_id;

    IF v_status != 'ACTIVE' THEN
        RAISE_APPLICATION_ERROR(-20001, 'Rezerwacja została już zakończona lub anulowana!');
    END IF;

    FOR item IN c_items LOOP
        v_qty_to_process := item.Quantity;

        FOR inv_batch IN c_inventory(item.Product_ID) LOOP
            EXIT WHEN v_qty_to_process <= 0;

            v_deduct := LEAST(v_qty_to_process, inv_batch.Reserved_Quantity);

            IF p_action = 'COMPLETED' THEN -- ugotowano
                UPDATE Inventory
                SET Quantity = Quantity - v_deduct,
                    Reserved_Quantity = Reserved_Quantity - v_deduct
                WHERE CURRENT OF c_inventory;
```

```
        ELSIF p_action = 'CANCELLED' THEN -- anulowano
            UPDATE Inventory
            SET Reserved_Quantity = Reserved_Quantity - v_deduct
            WHERE CURRENT OF c_inventory;
        END IF;

        v_qty_to_process := v_qty_to_process - v_deduct;
    END LOOP;
END LOOP;

-- aktualizacja
UPDATE Reservations
SET Status = p_action
WHERE Reservation_ID = p_reservation_id;

COMMIT; -- zapisujemy

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20002, 'Nie znaleziono takiej rezerwacji!');
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;

END;
```

- Przetwarzanie statusu rezerwacji składników po podjęciu decyzji przez kucharza (gotowanie lub anulowanie posiłku).
- Procedura przyjmuje jako parametry identyfikator rezerwacji oraz typ akcji (**COMPLETED** lub **CANCELLED**).
 - W przypadku **COMPLETED** (danie ugotowane) – system zmniejsza fizyczną ilość produktów (**Quantity**) w magazynie oraz zdejmuje rezerwację (**Reserved_Quantity**).
 - W przypadku **CANCELLED** (anulowanie) – produkty wracają do puli ogólnodostępnej (zmniejszane jest tylko **Reserved_Quantity**).
- Procedura operuje na kursorach z klauzulą **FOR UPDATE** (blokowanie wierszy na czas transakcji) i zdejmuje produkty według zasady FIFO (z partii o najkrótszej dacie ważności). Całość zabezpieczona jest instrukcjami **COMMIT** i **ROLLBACK**.
- Automatyzuje proces wydań magazynowych i zapobiega powstawaniu błędów (np. ujemnych stanów magazynowych).

Procedura rezerwacji składników: **Reserve_Dish**

```
CREATE OR REPLACE PROCEDURE Reserve_Dish (  
    p_User_ID IN NUMBER,  
    p_Dish_ID IN NUMBER  
) AS  
    v_Has_Enough_Stock BOOLEAN := TRUE;  
    v_Reservation_ID NUMBER;  
    v_Available_Qty NUMBER;  
  
    -- Wyciągam składniki potrzebne do zrobienia tego konkretnego dania  
    CURSOR c_Recipe IS  
        SELECT Product_ID, Required_Quantity  
        FROM Recipes  
        WHERE Dish_ID = p_Dish_ID;  
  
    -- Tabele pomocnicze, żeby zapamiętać w pamięci sesji co musimy zablokować i ile tego potrzebujemy  
    TYPE t_Product_ID IS TABLE OF Recipes.Product_ID%TYPE;  
    TYPE t_Required_Qty IS TABLE OF Recipes.Required_Quantity%TYPE;  
    v_Prod_IDs t_Product_ID := t_Product_ID();  
    v_Req_Qties t_Required_Qty := t_Required_Qty();  
BEGIN  
    -- BLOKOWANIE I WERYFIKACJA ASORTYMENTU  
    -- Przechodzę po kolei przez każdy składnik z przepisu i blokuję go w bazie (FOR UPDATE),  
    -- żeby w tym samym czasie nikt inny nie podkradł nam tych produktów ze spiżarni.  
    FOR r IN c_Recipe LOOP  
        v_Prod_IDs.EXTEND;  
        v_Req_Qties.EXTEND;  
        v_Prod_IDs(v_Prod_IDs.LAST) := r.Product_ID;  
        v_Req_Qties(v_Req_Qties.LAST) := r.Required_Quantity;  
  
        -- Liczę, ile w ogóle mamy wolnego i ważnego produktu we wszystkich partiach.  
        SELECT NVL(SUM(Quantity - Reserved_Quantity), 0)  
        INTO v_Available_Qty  
        FROM Inventory
```

```
WHERE Product_ID = r.Product_ID
AND Status = 'AVAILABLE'
AND Expiration_Date >= TRUNC(SYSDATE);

-- Jeśli w spiżarni brakuje chociaż jednego składnika, zaznaczam to sobie na później.
IF v_Available_Qty < r.Required_Quantity THEN
    v_Has_Enough_Stock := FALSE;
END IF;
END LOOP;

-- DECYZJA O TRANSAKCJI
IF NOT v_Has_Enough_Stock THEN
    -- Jeśli zabrakło jakiegoś składnika, wycofuję wszystkie blokady (ROLLBACK) i rzucam błąd.
    ROLLBACK;
    RAISE_APPLICATION_ERROR(-20001, 'Brak wystarczającej ilości składników w spiżarni do przygotowania tego dania!');
ELSE
    -- Jak wszystko jest na stanie, to tworzę główny wpis rezerwacji.
    INSERT INTO Reservations (User_ID, Dish_ID, Reservation_Date, Status)
    VALUES (p_User_ID, p_Dish_ID, SYSDATE, 'ACTIVE')
    RETURNING Reservation_ID INTO v_Reservation_ID;

    -- Teraz po kolei rezerwuję potrzebną ilość każdego składnika w partiach.
    FOR i IN 1..v_Prod_IDs.COUNT LOOP
        -- Zapisuję szczegóły, co dokładnie i w jakiej ilości zostało zarezerwowane dla danej rezerwacji.
        INSERT INTO Reservation_Items (Reservation_ID, Product_ID, Quantity)
        VALUES (v_Reservation_ID, v_Prod_IDs(i), v_Req_Qties(i));

        -- Muszę rozdysponować zapotrzebowanie na partie w magazynie.
        -- Zaczynam od tych partii, które mają najkrótszą datę ważności (FIFO dla przeterminowania).
        DECLARE
            v_Remaining_To_Reserve NUMBER := v_Req_Qties(i);
            v_Batch_ID NUMBER;
            v_Batch_Free_Qty NUMBER;

            CURSOR c_Batches IS
                SELECT Batch_ID, (Quantity - Reserved_Quantity) AS Free_Qty
```



```
FROM Inventory
WHERE Product_ID = v_Prod_IDs(i)
  AND Status = 'AVAILABLE'
  AND Expiration_Date >= TRUNC(SYSDATE)
ORDER BY Expiration_Date ASC
FOR UPDATE; -- Blokuje te rekordy żeby nikt inny ich nie użył
BEGIN
  OPEN c_Batches;
  LOOP
    FETCH c_Batches INTO v_Batch_ID, v_Batch_Free_Qty;
    EXIT WHEN c_Batches%NOTFOUND OR v_Remaining_To_Reserve = 0;

    -- Jeśli ta partia ma więcej wolnego niż potrzebuję, rezerwuję całość i kończę.
    IF v_Batch_Free_Qty >= v_Remaining_To_Reserve THEN
      UPDATE Inventory
      SET Reserved_Quantity = Reserved_Quantity + v_Remaining_To_Reserve
      WHERE Batch_ID = v_Batch_ID;
      v_Remaining_To_Reserve := 0;
    ELSE
      -- Jeśli partia ma za mało, rezerwuję ile się da i szukam w kolejnej partii.
      UPDATE Inventory
      SET Reserved_Quantity = Reserved_Quantity + v_Batch_Free_Qty
      WHERE Batch_ID = v_Batch_ID;
      v_Remaining_To_Reserve := v_Remaining_To_Reserve - v_Batch_Free_Qty;
    END IF;
  END LOOP;
  CLOSE c_Batches;
END;

-- Wrzucam info do logów, że dany użytkownik zablokował składniki.
INSERT INTO Inventory_Log (User_ID, Product_ID, Action_Type, Quantity_Change)
VALUES (p_User_ID, v_Prod_IDs(i), 'RESERVATION_ADD', v_Req_Qties(i));
END LOOP;

-- Wszystko się udało, zapisuję transakcję i puszczam blokady.
COMMIT;
```

```

    END IF;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK; -- W razie nieoczekiwanego błędu bazy danych wycofaj wszystko
        RAISE;
END Reserve_Dish;
/

```

- **Bezpieczeństwo współbieżności:** Procedura realizuje zaawansowany mechanizm rezerwacji składników pod konkretne danie. Używa blokady pesymistycznej (**FOR UPDATE**), co gwarantuje, że inny kucharz nie zużyje tych samych produktów w tym samym ułamku sekundy.
- **Rozliczanie po partiach:** Przydzielanie rezerwacji z poszczególnych partii odbywa się elastycznie z użyciem strategii FIFO – w pierwszej kolejności zawsze alokowane są najstarsze partie o najkrótszej dacie ważności, co wspiera model First-In First-Out.
- Jeśli w połowie weryfikacji algorytm zorientuje się, że brakuje choć jednego składnika na przygotowanie całego dania, następuje momentalne wycofanie wszystkich blokad (**ROLLBACK**), a błędna rezerwacja nie dochodzi do skutku. Oszczędza to czas kucharza i zapobiega niesłusznemu zamrażaniu surowców.

Procedura przekazania żywności: **Donate_Expired_Food**

```

CREATE OR REPLACE PROCEDURE Donate_Expired_Food (
    p_Admin_User_ID IN NUMBER
) AS
    v_User_Role VARCHAR2(20);
BEGIN
    -- Upewniam się, czy to na pewno Szef Kuchni (CHEF) odpala procedurę. Zwykły pracownik nie ma uprawnień.
    SELECT Role INTO v_User_Role FROM Users WHERE User_ID = p_Admin_User_ID;
    IF v_User_Role != 'CHEF' THEN
        RAISE_APPLICATION_ERROR(-20002, 'Tylko Szef Kuchni może uruchomić procedurę końca dnia!');
    END IF;

    -- Szukam partii jedzenia, które kończą ważność dzisiaj lub jutro i nie są jeszcze zarezerwowane.
    -- Wszystko to, co jest wolne (Quantity - Reserved_Quantity > 0), oddajemy na cele charytatywne.
    FOR r IN (
        SELECT Batch_ID, Product_ID, (Quantity - Reserved_Quantity) AS Available_To_Donate

```

```
FROM Inventory
WHERE Expiration_Date <= TRUNC(SYSDATE) + 1
      AND Status = 'AVAILABLE'
      AND (Quantity - Reserved_Quantity) > 0
) LOOP
  -- Zapisuję do logów, ile dokładnie oddaliśmy danej rzeczy na charytatywność (dlatego ujemna wartość).
  INSERT INTO Inventory_Log (User_ID, Product_ID, Action_Type, Quantity_Change)
  VALUES (p_Admin_User_ID, r.Product_ID, 'CHARITY_DONATION', -r.Available_To_Donate);

  -- Zeruję wolne zapasy z tej partii (zostaje tylko to, co już było wcześniej zarezerwowane).
  -- Jeśli po tym zabiegu nic nie zostało zarezerwowane (Reserved_Quantity = 0), oznaczam partię jako całkiem oddaną
  ('DONATED').
  UPDATE Inventory
  SET Quantity = Reserved_Quantity,
      Status = CASE WHEN Reserved_Quantity = 0 THEN 'DONATED' ELSE 'AVAILABLE' END
  WHERE Batch_ID = r.Batch_ID;
END LOOP;

-- Zapisuję zmiany na stałe.
COMMIT;
END Donate_Expired_Food;
/
```

- **Zarządzanie Zero-Waste na koniec dnia:** Codzienna procedura wywoływana przez Szefa Kuchni, która skanuje magazyn w celu znalezienia wszystkich partii surowców, których termin przydatności kończy się najpóźniej jutro, a które nie zostały w 100% użyte do rezerwacji dań.
- Uruchomienie procedury natychmiastowo odprowadza takie wolne zasoby ze stanu magazynu (ustawiając parametr **Quantity** równo z zablokowanymi zapasami i nadając partii końcowy status **'DONATED'**) oraz ściśle dokumentuje ten zaszczytny proces przekazania w tabeli logów.

Widoki

Widok raportujący: **V_Cook_Today**

```
CREATE OR REPLACE VIEW V_Cook_Today AS
WITH Available_Stock AS (
    -- dostępną ilość na półce (minus rezerwacje)
    SELECT
        Product_ID,
        SUM(Quantity - Reserved_Quantity) AS Total_Available,
        MIN(Expiration_Date) AS Soonest_Exp_Date
    FROM Inventory
    WHERE Status = 'AVAILABLE'
    AND Expiration_Date >= SYSDATE
    GROUP BY Product_ID
),
Dish_Capabilities AS (
    -- łączymy dania z ich przepisami i naszym magazynem
    SELECT
        d.Dish_ID,
        d.Dish_Name,
        d.Category,
        MIN(FLOOR(NVL(st.Total_Available, 0) / r.Required_Quantity)) AS Max_Portions,
        MIN(st.Soonest_Exp_Date) AS Critical_Expiration_Date -- data ważności
    FROM Dishes d
        JOIN Recipes r ON d.Dish_ID = r.Dish_ID
        LEFT JOIN Available_Stock st ON r.Product_ID = st.Product_ID
    GROUP BY d.Dish_ID, d.Dish_Name, d.Category
)

SELECT
    Dish_Name AS "Nazwa Dania",
    Category AS "Kategoria",
    Max_Portions AS "Możliwych Porcji",
    Critical_Expiration_Date AS "Najpilniejszy Składnik"
FROM Dish_Capabilities
WHERE Max_Portions > 0
ORDER BY Critical_Expiration_Date ASC;
```

- Dynamiczne wspieranie decyzji Szefa Kuchni i kucharzy poprzez odpowiedź na pytanie: "Co możemy w tym momencie ugotować z wolnych składników?".
- Widok jest złożonym zapytaniem analitycznym, które wykorzystuje podzapytania (klauzula **WITH**), złączenia wielotabelowe (**JOIN**), funkcje agregujące (**SUM**, **MIN**) oraz funkcję matematyczną **FLOOR**. Zapytanie wylicza realną ilość dostępnych produktów (fizyczny stan minus rezerwacje innych kucharzy), zestawia je z wymaganiami z przepisów (**Recipes**) i oblicza maksymalną liczbę pełnych porcji, jaką można przygotować. Wyniki są filtrowane (tylko dania, na które starczy składników) i sortowane według **daty ważności najpilniejszego składnika** (promowanie dań z produktów, które psują się najszybciej).
- Kluczowe narzędzie w strategii *Zero Waste* – pozwala kucharzowi szybko podjąć decyzję o przygotowaniu dań ze składników, które w przeciwnym razie musiałyby zostać wyrzucone lub oddane.

Indeksy i Optymalizacja

```
-- Indeks do szybkiego wyszukiwania psującego się jedzenia
CREATE INDEX idx_inventory_exp_date ON Inventory(Expiration_Date);

-- Indeksy na klucze obce
CREATE INDEX idx_inventory_product_id ON Inventory(Product_ID);
CREATE INDEX idx_recipes_dish_id ON Recipes(Dish_ID);
CREATE INDEX idx_recipes_product_id ON Recipes(Product_ID);
CREATE INDEX idx_reservations_user_id ON Reservations(User_ID);
CREATE INDEX idx_res_items_res_id ON Reservation_Items(Reservation_ID);
CREATE INDEX idx_res_items_prod_id ON Reservation_Items(Product_ID);
CREATE INDEX idx_log_product_id ON Inventory_Log(Product_ID);
```

- Zaprojektowałyśmy zoptymalizowane indeksy bazodanowe (**B-Tree**) dla kolumn, po których najczęściej wykonujemy warunki ograniczające **WHERE** oraz na kluczowych kolumnach łączących tabele relacyjne w klauzulach **JOIN**.
- Znacząco przyspiesza to m.in. generowanie skomplikowanych widoków raportowych dla Szefa Kuchni.
- Dodatkowo konsekwentna indeksacja kluczy obcych zapobiega problematycznym blokadom pełnych tabel (tzw. *table locks*) podczas usuwania lub aktualizowania rekordów nadrzędnych.

Uprawnienia i Bezpieczeństwo (RBAC)

```
-- Tworzymy role biznesowe
CREATE ROLE ROLE_COOK;
CREATE ROLE ROLE_CHEF;

-- Zwykły kucharz: wgląd w zapasy i uruchamianie rezerwacji
GRANT SELECT ON Dishes TO ROLE_COOK;
GRANT SELECT ON Recipes TO ROLE_COOK;
GRANT SELECT ON Product_Catalog TO ROLE_COOK;
GRANT SELECT, UPDATE ON Inventory TO ROLE_COOK;
GRANT INSERT, SELECT, UPDATE ON Reservations TO ROLE_COOK;
GRANT INSERT, SELECT ON Reservation_Items TO ROLE_COOK;
GRANT EXECUTE ON Reserve_Dish TO ROLE_COOK;
GRANT EXECUTE ON Resolve_Reservation TO ROLE_COOK;

-- Szef Kuchni: dziedziczy po kucharzu, plus pełna edycja menu i zamknięcie dnia
GRANT ROLE_COOK TO ROLE_CHEF;
GRANT INSERT, UPDATE, DELETE ON Dishes TO ROLE_CHEF;
GRANT INSERT, UPDATE, DELETE ON Recipes TO ROLE_CHEF;
GRANT INSERT, UPDATE, DELETE ON Product_Catalog TO ROLE_CHEF;
GRANT EXECUTE ON Donate_Expired_Food TO ROLE_CHEF;
```

- Cała struktura autoryzacji do bazy danych została ukształtowana w oparciu o fundamentalną zasadę bezpieczeństwa: **najmniejszego uprzywilejowania** (ang. *Principle of Least Privilege*).
- **ROLE_COOK** (Kucharz) posiada jedynie bardzo restrykcyjne prawa odczytu do menu i stanów spiżarni. Co kluczowe – kucharz w ogóle nie ma prawa tworzyć ręcznie produktów, a stany spiżarni zmniejsza i rezerwuje wyłącznie wywołując w pełni ufany i obudowany logiką biznesową interfejs bazy, czyli stworzone procedury (**Reserve_Dish**). Uziemnia to praktycznie do zera potencjał omijania logowania akcji przez szeregowych pracowników.
- **ROLE_CHEF** (Szef Kuchni) dziedziczy podstawowe prawa zwykłego kucharza, ale dzięki awansowi otrzymuje pełne przywileje do wprowadzania nowych potraw na menu, modyfikacji starych przepisów i wreszcie posiada autoryzację administracyjną do uruchomienia procedury **Donate_Expired_Food** na poczet działań fundacyjnych.

Backend (API)

Do integracji bazy danych Oracle z aplikacją kliencką (frontendem w React) zastosowano nowoczesny, wysokowydajny framework **FastAPI** w języku Python. Backend pełni rolę bezpiecznego łącznika (REST API), który przyjmuje żądania z interfejsu użytkownika, mapuje je na zapytania relacyjne i przekazuje do wykonania po stronie serwera Oracle, a następnie zwraca ustrukturyzowane wyniki w formacie JSON.

Implementacja znajduje się w podfolderze backend w pliku `main.py`

Metoda	Ścieżka (URI)	Model wejściowy	Opis działania biznesowego	Powiązany obiekt bazy
GET	/api/reports/what-to-cook	Brak	Pobiera listę potraw, które kucharz może przygotować na bazie aktualnych, świeżych stanów spiżarni.	V_Cook_Today
POST	/api/reservations/reserve	ReserveRequest	Przesyła ID kucharza oraz ID dania. Blokuje zasoby w bazie i tworzy rezerwację.	Reserve_Dish
POST	/api/reservations/resolve	ResolveRequest	Aktualizuje stan rezerwacji – pomniejsza fizyczny stan magazynu w przypadku ugotowania potrawy lub zwraca surowce do puli wolnej w przypadku anulowania.	Resolve_Reservation
POST	/api/admin/end-of-day	EndOfDayRequest	Weryfikuje rolę użytkownika (wymagany Szef Kuchni) i automatycznie zeruje oraz przekazuje fundacjom towary bliskie przeterminowania.	Donate_Expired_Food
GET	/api/inventory	Brak	Zwraca pełną listę partii magazynowych wraz z nazwami produktów, ilościami wolnymi i zarezerwowanymi.	Inventory + Product_Catalog
GET	/api/reservations	Brak	Pobiera listę wszystkich aktywnych oraz zakończonych rezerwacji wraz z danymi kucharza i nazwą potrawy.	Reservations + Users + Dishes
GET	/api/users	Brak	Zwraca listę pracowników systemu wraz z ich rolami (CHEF/COOK), wykorzystywaną m.in. do autoryzacji sesji.	Users

Jak włączyć?

Przechodzimy do podfolderu z backendem:

```
cd backend
```

Uruchomienie:

```
uvicorn main:app --reload
```

Testowanie W przeglądarce wpisz: <http://127.0.0.1:8000/docs>

Frontend

Interfejs użytkownika (klient) został zrealizowany w formie nowoczesnej aplikacji typu Single Page Application (SPA) przy użyciu biblioteki **React.js**. Narzędziem używanym do budowania i serwowania aplikacji jest **Vite**, co zapewnia błyskawiczne środowisko deweloperskie oraz zoptymalizowany proces budowania.

Aplikacja frontendowa komunikuje się asynchronicznie z serwerem FastAPI za pośrednictwem zapytań RESTowych, wysyłając i pobierając dane w formacie JSON. Umożliwia ona w sposób intuicyjny przeglądanie stanów magazynowych spiżarni, zgłaszanie rezerwacji, a Szefowi Kuchni ułatwia weryfikację tego, co można dzisiaj ugotować z dostępnych składników. Warstwa wizualna została oparta o standardowy język CSS.

Uruchomienie aplikacji klienckiej

Przechodzimy do podfolderu z frontendem:

```
cd frontend
```

Instalacja zależności środowiska Node.js (wymagane tylko za pierwszym razem):

```
npm install
```


Uruchomienie serwera deweloperskiego:

```
npm run dev
```

Aplikacja będzie domyślnie dostępna w przeglądarce pod adresem, który zostanie wyświetlony w konsoli (najczęściej <http://localhost:5173/>).